

Enhancing Container Security Through Phase-Based System Call Filtering

Ke Chen^{ID}, Hui Lu^{ID}, Yinnan Yao, Binxing Fang, Yuan Liu^{ID}, *Senior Member, IEEE*, and Zhihong Tian^{ID}

Abstract—Container technology in cloud computing has improved resource utilization and deployment efficiency, but it also introduces new security risks. Excessive privileges in containerized environments can allow attackers to exploit insufficiently restricted system calls, potentially leading to container escapes and other attacks. Some system calls are only necessary during the initialization phase of a container, and allowing them during runtime can increase the risk of exploitation. This paper proposes a Phase-based System Call Filtering (PSF) method to minimize system call permissions during the runtime of cloud containers. The PSF method builds a comprehensive whitelist of system calls during the initialization phase to cover all necessary calls for containerized applications. During runtime, a refined, phase-specific system call whitelist is enforced, dynamically adjusting privileges based on the functions encapsulated within the container. Additionally, we introduce a container phase recognition algorithm to distinguish between initialization and runtime phases, supporting the generation of phase-specific system call lists. Experimental results show that the proposed method enhances runtime system call restrictions, minimizes privileges, and improves the overall security of cloud containers.

Index Terms—Containers, system call filtering, static analysis, dynamic analysis, phased.

I. INTRODUCTION

CLOUD computing has emerged as a critical solution in modern computing environments. With the rapid growth of cloud services and the increasing demands for data processing, traditional computing resources are increasingly inadequate

to meet the required flexibility and efficiency. As lightweight, portable, and manageable computing units, cloud environments offer enhanced resource utilization and support for distributed, dynamic computational needs, improving the scalability and flexibility of systems. By packaging applications and services into independent modules, cloud environments facilitate seamless operation across heterogeneous hardware and diverse environments. Despite these advantages, cloud environments face significant security challenges, particularly due to the shared use of the underlying operating system kernel, making them more susceptible to attacks compared to traditional virtual machines. [1].

The quintessential delineation between containers and virtual machines hinges on their conjoined host operating system kernel. This inherent shared architecture capacitates applications encapsulated within containers with privileged access to host-centric system resources, encompassing filesystems, networking paradigms, and process management. Notwithstanding the isolation and resource delineation afforded by the Linux kernel through namespace [2] and cgroup [3] frameworks, these foundational isolation paradigms are not impervious and are susceptible to certain security vulnerabilities, encompassing inter-process manipulations, privilege escalation stratagems, and inadvertent disclosure of sensitive information.

To enhance the security of containerized ecosystems, modern container platforms and orchestration frameworks have implemented a range of protective measures, including namespace isolation, filesystem partitioning, and network segmentation. Additionally, to bolster container resilience, Linux-based Mandatory Access Control (MAC) frameworks such as SELinux [4] and AppArmor [5], along with syscall filtering tools like Seccomp BPF [6], have gained widespread adoption. However, despite the robust security mechanisms provided by their default configurations [7], leading container frameworks such as Docker [8] and Kubernetes [9] remain vulnerable to latent security risks due to the complexity inherent in their configurations [10], [11], [12], [13], [14].

In this milieu, the crux of this research endeavors to harness an algorithmic Seccomp syscall filtration stratagem generation framework, aiming to curtail the prospective attack vectors of containers and mitigate the intricacies concomitant with manual policy articulation, thereby championing the ethos of minimal privilege deployment paradigms for containers. The salient contribution of this research is anchored in algorithmic policy synthesis, markedly attenuating the manual labor and inadvertent discrepancies associated with policy articulation, thereby

Received 27 December 2024; revised 18 June 2025; accepted 22 June 2025. Date of publication 26 June 2025; date of current version 8 September 2025. This work was supported in part by the National Key Research and Development Plan under Grant 2023YFB3107300, in part by the National Natural Science Foundation of China under Grant 62272121, Grant U2436208, and Grant 62372129, in part by Guangdong S&T Program under Grant 2024B0101010002, in part by the Project of Guangdong Key Laboratory of Industrial Control System Security under Grant 2024B1212020010, and in part by the Major Key Project of PCL under Grant PCL2024A05. Recommended for acceptance by T. Xiang. (Corresponding authors: Hui Lu; Yuan Liu.)

Ke Chen is with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China, also with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou 511370, China, and also with the Guangdong Key Laboratory of Industrial Control System Security, Huangpu Research School of Guangzhou University, Guangzhou 510530, China.

Hui Lu, Yinnan Yao, Binxing Fang, Yuan Liu, and Zhihong Tian are with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou 510530, China, and also with the Guangdong Key Laboratory of Industrial Control System Security, Huangpu Research School of Guangzhou University, Guangzhou 510530, China (e-mail: luhui@gzhu.edu.cn; yaoyinnan@gzhu.edu.cn; fangbx@cae.cn; yuanliu@gzhu.edu.cn; tianzhihong@gzhu.edu.cn).

Digital Object Identifier 10.1109/TCC.2025.3583414

amplifying the overarching security postures of platforms such as Docker and Kubernetes.

Traditional technical studies primarily focus on obtaining system call policies through static or dynamic analyses. However, they often overlook the differential system call usage patterns during the initialization and runtime phases of containers. Middleware containers, exemplified by MySQL, Redis, and Apache, may invoke system calls like `fork()` and `mount()` during their initialization processes. Although these system calls are infrequently utilized during runtime, they still present potential security risks.

In elucidating these intricacies, the seminal innovations encapsulated within this research are:

- We propose a phase-based system call filtering framework for cloud containers that dynamically adjusts system call policies according to the container's execution phase (initialization vs runtime). Unlike prior works that generate static policies based on collected syscall sets, our approach supports live switching of syscall policies at runtime based on phase recognition, thereby minimizing the container's attack surface dynamically.
- We design and implement a container phase recognition algorithm that detects the transition between initialization and runtime phases, and triggers corresponding Seccomp policy updates in real time. This enables practical integration of phase-based syscall filtering in cloud-native platforms such as Docker and Kubernetes.
- We develop a fully integrated solution with experimental validation on realistic workloads (e.g., Redis, MySQL) within Docker and Kubernetes environments. Our results demonstrate that the proposed method achieves fine-grained syscall control, significantly reduces unnecessary privileges, and introduces minimal performance overhead while enhancing container security.

II. BACKGROUND

A. Container Technology

Container technology is a lightweight virtualization technique that packages applications and their dependencies into a portable container, allowing them to be deployed and run in various computing environments to establish a runtime environment for software. Container technology offers advantages such as high resource utilization, rapid deployment, and ease of management, making it widely utilized in cloud computing and large-scale application deployments.

However, container technology also presents certain security challenges. Firstly, although containerization isolates applications and their dependencies, they still share the same underlying operating system kernel. As a result, malicious code within a container may potentially impact the host system and other containers. Secondly, the network isolation provided by containers is not absolute; containers can communicate with each other, creating potential pathways for network-based attacks to propagate between containers. Additionally, container technology is susceptible to other security issues, such as improper permission

management and the inclusion of malicious software within container images.

To address these challenges, it is imperative to incorporate additional security mechanisms into the design and implementation of container technology, such as enhanced network isolation, resource constraints, and permission management. Simultaneously, implementing finer-grained system call controls for processes within containers is also a crucial approach to enhancing container security.

B. System Call

System calls are interfaces provided by the operating system for user programs, allowing them to request access to underlying hardware and system resources. Common system calls include file operations, process management, network communication, and memory management, among others. The execution of a system call typically involves transitioning from user mode to kernel mode and performing privileged operations.

In the context of container technology, the isolation and security of containers largely depend on the control of system calls within the container. For instance, certain sensitive system calls can be restricted or prohibited to prevent malicious containerized programs from launching attacks through these interfaces. Simultaneously, improper restrictions on system calls may result in applications failing to operate correctly; hence, compatibility must be maintained while ensuring security.

Specifically, the system calls relevant to containers include, but are not limited to, file operations, process management, network communication, and memory management. Some sensitive system calls, such as `execve()`, `ptrace()`, `mmap()`, and `socket`, can be exploited for attacks, such as executing malicious code, debugging other processes, or conducting port scanning. Therefore, restricting these system calls is a crucial measure for enhancing container security.

In containerized environments, due to their unique isolation and encapsulation properties, certain system calls may be invoked more frequently by applications within containers than in conventional non-containerized settings, where they might be rarely or never invoked. As shown in the Table I are some system calls that may be specifically called in certain scenarios in a container environment.

To address the security issues posed by system calls in container technology, mechanisms like Seccomp can be employed to restrict the system calls within containers. For instance, one can limit a containerized program's access to specific system calls or constrain the parameters of system calls. These control strategies must take into account the runtime environment and requirements of containerized programs, as well as the actual usage patterns of system calls, to achieve effective security protection.

C. Seccomp

Secure Computing Mode (Seccomp) is a security mechanism in the Linux kernel designed to restrict the system calls that a

TABLE I
CONTAINER RELATED SYSTEM CALLS

Category	System Call	Description	Common Use Cases
Filesystem Isolation & Mounting	pivot_root()	Modifies the root filesystem of a process.	Used during container initialization for filesystem isolation.
	mount()	Mounts a filesystem.	Used to mount filesystems within a container's isolated environment.
Network Isolation & Configuration	setns()	Allows a process to join or create a new network namespace.	Used for network isolation within containers.
	clone(CLONE_NEWNET)	Creates a new network namespace.	Used for network isolation within containers.
Process Isolation & Control	unshare()	Detaches a process from certain namespaces (e.g., network, mount points).	Provides process-level isolation.
Resource Limitation & Control	setrlimit()	Sets resource limits for a process.	Controls resource usage within containers.
	getrlimit()	Retrieves resource limits for a process.	Queries resource limits for processes within containers.
cgroup Related Calls	mkdir()	Creates a cgroup directory.	Creates directories for control groups (cgroups).
	mount() (cgroup related)	Mounts the cgroup filesystem.	Activates the cgroup configuration.
	write() (cgroup related)	Writes data to a cgroup control file.	Updates configuration information for cgroups.
Permissions & Security	setuid()	Changes the user ID of a process.	Enhances permission isolation within containers.
	setgid()	Changes the group ID of a process.	Enhances permission isolation within containers.
	seteuid()	Changes the effective user ID of a process.	Enhances permission isolation within containers.
	setegid()	Changes the effective group ID of a process.	Enhances permission isolation within containers.

TABLE II
SECURITY ANALYSIS OF SYSTEM CALLS BLOCKED BY DEFAULT IN SECCOMP BPF

System Call	Security Rationale
add_key	Prevents a container from using kernel keyrings in unnamed namespaces.
adjtimex	Similar to clock_settime and settimeofday; time/date is not namespaced. Also governed by CAP_SYS_TIME.
BPF	Refuses to load potentially persistent BPF programs into the kernel. Governed by CAP_SYS_ADMIN.
clone	Refuses to clone new namespaces. Governed by CAP_SYS_ADMIN for CLONE_* flags, excluding CLONE_USERSNS.
ioperm	Prevents a container from modifying kernel I/O privilege levels. Governed by CAP_SYS_RAWIO.
IOPL	Prevents a container from modifying kernel I/O privilege levels. Governed by CAP_SYS_RAWIO.
ptrace	Traces/analyses system calls which could leak a lot of information from the host. Blocked by dropping CAP_PTRACE.
quotactl	Quota system call that allows a container to disable its resource limits or process accounting. Governed by CAP_SYS_ADMIN.
reboot	Prevents a container from restarting the host. Governed by CAP_SYS_BOOT.
socket, socketpair	Used for sending or receiving packets and other socket operations. All socket calls and socketpair calls are blocked except for AF_UNIX, AF_INET, AF_INET6, AF_NETLINK, and AF_PACKET.
stime	Time/date is not namespaced. Governed by CAP_SYS_TIME.

process can make, thereby reducing its attack surface. Abbreviated as Seccomp, Seccomp BPF is an extension of the Seccomp mechanism that allows for the control of system calls through the authoring of Berkeley Packet Filter (BPF) programs. Unlike traditional system call filters, Seccomp filters operate using a dynamic whitelist approach, permitting processes to invoke only the pre-defined system calls while blocking all others. This design prevents attackers from exploiting vulnerabilities within a process to invoke unauthorized system calls, thereby enhancing system security. However, crafting Seccomp system call filtering policies manually is highly challenging, requiring an in-depth understanding of both system calls and application behaviors, posing a significant barrier for many developers.

Compared to the traditional Seccomp mechanism, Seccomp BPF offers more flexible control over system calls. By employing BPF programs for filtering, it enables granular control over system call parameters, return values, invocation counts, and more. Additionally, Seccomp BPF boasts good performance and compatibility, finding widespread adoption in areas such as containerization and cloud computing.

Although Docker and Podman ship with a default Seccomp configuration file, there are some system calls that are whitelisted as shown in the Listing 1. The Table II below lists important (but not all) system calls that are effectively blocked because they are not in the whitelist. This table contains the reason why each system call is blocked instead of a whitelist. But these defaults allow a wide range of system calls, which may not be enough as a strong security measure. However, a significant drawback of Seccomp BPF is its challenging manual configuration. Given that Seccomp BPF requires writing BPF programs for system call filtering, writing and debugging such programs requires specialized skills and experience. Additionally, configuring Seccomp BPF needs to be considered based on the actual usage patterns and requirements of the containerized application, further requiring domain-specific knowledge.

To alleviate the challenges associated with Seccomp BPF configuration, technologies that automate the generation of Seccomp BPF programs can be leveraged to produce filtering policies automatically. By analyzing the code and runtime environments of containerized applications, these technologies

Listing 1: Default Seccomp File.

```

{
  "defaultAction": "SCMP_ACT_ERRNO",
  "defaultErrnoRet": 1,
  "archMap": [
    {
      "architecture": "SCMP_ARCH_X86_64",
      "subArchitectures": [
        "SCMP_ARCH_X86",
        "SCMP_ARCH_X32"
      ]
    },
    ],
  "syscalls": [
    {
      "names": [
        "accept",
        "accept4",
        "access",
        "adjtimex",

```

can autonomously generate appropriate Seccomp BPF filtering policies, thereby enhancing both the efficiency and security of Seccomp BPF utilization.

D. eBPF

Extended Berkeley Packet Filter (eBPF) is a virtual machine technology within the Linux kernel that enables the execution of user-defined programs in the kernel space. Initially proposed by Daniel Bovet and Rinaldo Ronzini in 2008, eBPF has evolved into a programmable, high-performance kernel extension technology.

In the realm of container security, eBPF offers several advantages. Firstly, it enables secure control over various operations such as system calls, networking, and file systems through the authoring of BPF programs. These BPF programs can be dynamically loaded at runtime, facilitating real-time protection of containerized applications. Secondly, eBPF operates with high efficiency, capable of delivering robust security controls without compromising container performance. Additionally, eBPF supports multiple programming languages, including C, Go, and Rust, allowing developers to choose languages that best suit their preferences.

Furthermore, eBPF supports integration with other container security technologies. For instance, combining eBPF with Seccomp BPF can achieve dual-layer security protection for containerized applications. This integrated approach can selectively restrict system calls based on enhanced observability of container runtime behavior.

As a programmable kernel extension technology, eBPF holds significant promise in the field of container security. Its efficiency, flexibility, and scalability provide a more reliable and efficient safeguard for container security.

III. RELATED WORK

In the realm of container technology security, researchers have proposed various strategies and tools to mitigate potential kernel attack risks faced by containers. Among them, gVisor [15] and Kata Containers [16] are two notably representative solutions. gVisor employs a user-space implementation, offering a more streamlined subset of Linux system call interfaces and intercepting and scrutinizing system calls through the use of Ptrace technology. In contrast, Kata Containers adopts a container virtualization approach, utilizing a virtual machine as a Pod within Kubernetes to achieve a higher level of container isolation. Although Kata Containers has made significant optimizations in areas such as simulating hot-plug and I/O devices and conducted thorough pooling of VMM and Cgroups [17], the isolation provided through its virtual machine manager often results in some performance overhead, particularly concerning I/O operations.

In subsequent research, scholars like Kurmus et al. introduced the KTrim method [18], aiming to eliminate unused Linux kernel functionalities within containers. This method leverages kernel-based triggers known as Kprobes [19] to analyze container startup and runtime phases, determining which kernel functionalities are not invoked by applications and restricting these functionalities at runtime using the Kprobes technology.

Researchers like Jang et al. conducted an in-depth quantitative assessment of the potential risks associated with system calls within containers [20], particularly focusing on runc [21], gVisor, and Kata Containers. Their study emphasizes quantifying the number of system calls that containers might expose under specific configurations and evaluates the potential impact of such exposures on system security.

It is worth noting that in recent years, an increasing body of research has begun to focus on methods for automatically generating Seccomp system call filtering policies, aiming to further enhance container security.

Existing research on Seccomp syscall policy generation can be categorized into static and dynamic analysis. Static analysis generates policies without executing the program by analyzing source code or binaries to identify required system calls [22], [23], [24], [25], [26]. Confine [27] builds a call graph combining direct and libc-mediated calls to improve accuracy, though it may include redundant calls. Sysfilter [28], using binary-only analysis with Nibbler [29], removes more unnecessary calls than Confine. Abhaya [30] generates policies from source code, while Decap infers required capabilities [31] based on privilege reduction. Ghavamnia et al. [32] divide syscall needs into initialization and runtime phases by analyzing the call graph, but their method relies on manually defined boundaries and is limited to common containers. While static analysis works without runtime environments, it may miss calls only triggered during execution. In addition, standard libraries such as glibc or musl can introduce redundant calls, and different programming languages require different analyzers, increasing implementation complexity.

Dynamic analysis tools like strace [33], Sysdig [34], and eBPF [35] collect syscall traces during actual execution, allowing direct observation of real behavior and compensating for

limitations of static methods. Slim [36] uses ptrace to generate policies and reduce image size. Lei et al. [37] observed differences in syscall behavior across phases and proposed generating separate policies for initialization and execution. Pancholi et al. [38] integrated ASan [39] to detect runtime vulnerabilities and enforce policy adaptation. Iguni et al. [40] proposed Sprofiler, combining static analysis with eBPF-based dynamic capture. Kim et al. [41] and Wang et al. [42] further explored hybrid approaches to produce more precise syscall policies. Although dynamic analysis provides detailed and real-time data, it requires execution in a real environment, consumes more resources, and may still miss rare or complex execution paths.

Most existing methods rely on static policy generation and lack the capability to dynamically adapt to variations in system call behavior across different stages of the container lifecycle (e.g., initialization and runtime). As a result, these methods struggle to detect behavioral anomalies at different stages and often produce policies that are either overly permissive or fail to accurately reflect the application's actual behavior during distinct phases [43], [44]. Although SYSPAR [45] has demonstrated an automated system call filtering mechanism based on execution stage differentiation, its target is limited to individual binary programs. In contrast, container environments exhibit far greater complexity: they typically involve multiple executables—including main service processes, initialization scripts, and auxiliary tools—as well as multi-process coordination, dynamic loading, and internal task scheduling. These characteristics make container execution substantially more intricate than that of a single binary, thereby limiting the direct applicability of SYSPAR's approach in containerized scenarios.

To address this limitation, we propose a stage-aware system call whitelist generation method. During the initialization phase, our method produces a comprehensive whitelist that covers all system calls required for container bootstrapping. During the runtime phase, a more fine-grained, phase-specific whitelist is enforced, dynamically adjusting system call permissions based on the actual functionality executed within the container to achieve a higher degree of privilege minimization.

Furthermore, we introduce a container stage recognition algorithm capable of accurately distinguishing between initialization and runtime phases, thereby supporting the generation of phase-specific syscall policies. This approach not only enhances the specificity and precision of syscall filtering but also effectively reduces the attack surface and strengthens the security posture of containers during runtime.

IV. PHASED SYSCALL FILTER (PSF)

We introduce the Phased Syscall Filter (PSF), a dynamic system call analyzer based on eBPF technology, specifically designed for monitoring dynamic behaviors during container runtime.

The dynamic analyzer of PSF was crafted with a focus on performance optimization while balancing resource consumption. By harnessing the efficiency of eBPF technology, we can execute code in kernel space, effectively minimizing the frequent

transitions between user space and kernel space. Moreover, by employing efficient data structures and algorithms, we aim to minimize the utilization of system resources.

The PSF employs a system call filtering strategy generation method that combines the results from both static and dynamic analyses. Initially, potential system calls required by the program are identified using static analysis tools. Subsequently, the actual system call requirements during program runtime are captured through dynamic analysis. By integrating the findings from both methods, a comprehensive whitelist of system calls for the initialization phase is generated. This approach not only offers a holistic view of system call requirements but also provides a robust foundation for subsequent filter rule designs.

During the container runtime, to precisely differentiate between the initialization and execution phases of system calls, PSF introduces a container segmentation point identification algorithm. This algorithm utilizes techniques such as time-window segmentation and similarity extraction to accurately pinpoint the transition points between these two crucial phases, thereby establishing clear temporal benchmarks for subsequent system call filtering and monitoring.

Through its thorough dynamic analysis and efficient filtering mechanisms, PSF furnishes containers with a robust and detailed system call management and monitoring tool, aiming to bolster both the security and performance of containers. In the following sections, we will delve into the details of PSF in four parts.

A. Dynamic System Call Analysis Via eBPF

The system call dynamic analyzer performs dynamic analysis by continuously recording detailed information about system calls while the container is running. The eBPF(Extended Berkeley Packet Filter) technology is specialized in real-time monitoring of system call executions. This technology permits the insertion of custom code within the kernel to perform specific actions upon the occurrence of system calls.

To further elaborate on the working mechanism, we formalize each captured system call as a tuple:

$$SC_t = \langle PID, comm, SYSCALL, ARGS, RET, TS \rangle \quad (1)$$

where *PID* denotes the process identifier, *comm* the command, *SYSCALL* the system call name (e.g., *openat*, *execve*), *ARGS* the argument list, *RET* the return value, and *TS* the timestamp. These tuples are recorded by eBPF programs attached to kernel tracepoints or kprobes.

The analyzer uses a ring buffer and eBPF maps to facilitate efficient communication between kernel and user space. Ring buffers ensure lock-free transmission, minimizing latency. By embedding decision logic within the kernel and decoupling bulk analysis to user space, we ensure near-real-time responsiveness while maintaining system stability and resource constraints.

In our system, the automatic identification of execution phases is implemented using dynamic tracing capabilities provided by eBPF. Specifically, during container execution, we deploy eBPF programs to intercept critical system calls and extract contextual information related to each invocation, which is then used to determine its corresponding behavioral phase. The

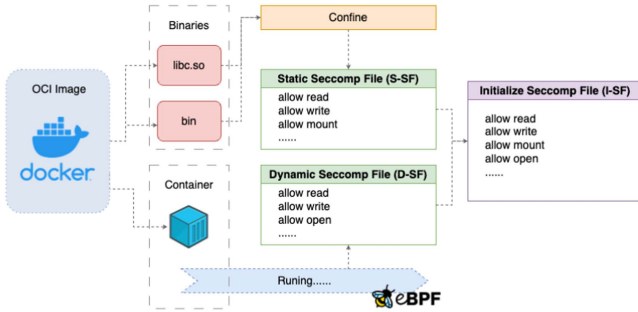


Fig. 1. Initialize seccomp file.

extracted context includes system call identifiers, parameters (e.g., file paths, network addresses), process names (`comm`), parent process identifiers (PPID), and command-line arguments. Based on these features and a predefined set of heuristic rules associated with typical phase patterns, the system classifies calls into different phases such as *initialization*, *setup*, and *runtime*.

For instance, during the initialization phase, the container typically invokes system calls such as `mount`, `pivot_root`, and `sethostname`, often triggered by processes like `runc`, `docker-init`, or user-defined entrypoint scripts. By identifying these behavioral patterns and their execution context, the system can reliably associate the activity with the initialization phase. As the container transitions to the runtime phase, user-defined processes (e.g., web servers or database services) dominate the syscall behavior, and the system classifies subsequent calls accordingly. This mechanism operates without modifying container images or requiring explicit in-app annotations, providing a general and automated solution for phase identification.

Considering that monitoring or alerting for all unrestricted system calls might inundate security operations personnel with excessive log alerts, hindering the accurate analysis of potential attack vectors, it is imperative to implement targeted monitoring and alerts for risky system calls during the runtime phase of containers.

B. Initialization-Phase System Call Filter Generation

The system call whitelist generation mechanism is one of the core functionalities of this approach. It integrates both static and dynamic analysis methods to produce a system call whitelist that encapsulates results from both analyses. Through this reasonable combination, a more comprehensive system call whitelist can be obtained. Static analysis can make the whitelist relatively comprehensive, but it lacks some system calls used at runtime, which can be supplemented by dynamic analysis. In this chapter, we will provide a detailed overview of the workflow for both static and dynamic analyses, and the overall architecture is shown in Fig. 1.

1) *Static Analysis*: During the static analysis phase, we begin by examining the program's source or binary code to identify potential system calls it might require. This encompasses both system calls invoked directly within the program and those invoked indirectly through library functions. We utilize the static analysis tool, *Confine*, to construct a call graph of the

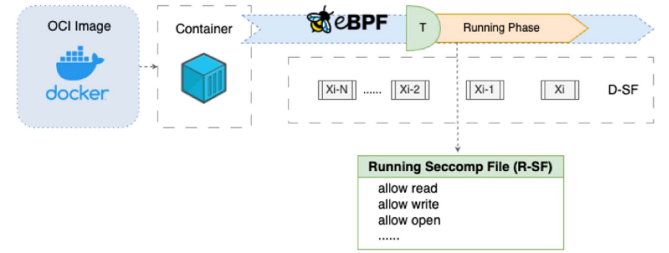


Fig. 2. Segment point identification.

program. This enables us to understand the program's system call requirements under various scenarios, forming the Static Seccomp File (S-SF) for system call filtering based on static analysis.

2) *Dynamic Analysis*: Although static analysis provides an initial system call whitelist, potential errors and program crashes may still arise in practical usage due to incomplete coverage of legitimate system call behaviors. To address this limitation and enhance the comprehensiveness of the whitelist, we propose a dynamic supplementation method based on a crash information loop during dynamic analysis. In this method, detailed data regarding each system call—including its parameters, return values, and invocation context—is collected through eBPF instrumentation. In addition, crash information resulting from missing system calls that trigger program failures is systematically recorded and analyzed.

A mapping table is constructed within the crash information loop to correlate missing system calls with specific crash signatures. By establishing this mapping, missing system calls that are essential for correct execution can be accurately identified and incorporated into the whitelist in a controlled manner. The outcome is the generation of a dynamic Seccomp file (D-SF), which reflects the system call requirements ascertained through dynamic analysis.

To ensure that the generated syscall filtering policies satisfy both the functional correctness of containerized applications and the principle of least privilege, the PSF framework integrates this dynamic enumeration mechanism exclusively within the policy generation phase. This process captures crashes resulting from insufficient system call permissions during both initialization and runtime stages and supplements the whitelist as needed. Importantly, dynamic enumeration is strictly confined to controlled development or testing environments, thereby producing reusable and reproducible syscall whitelists suitable for production deployment without introducing runtime policy relaxation or additional attack surfaces.

Finally, the D-SF is combined with the statically derived syscall whitelist (S-SF) to form a refined and comprehensive Initialization Seccomp file (I-SF), ensuring a precise and secure policy for the container's initialization phase.

3) *Initialize Seccomp File*: By combining the results from both static and dynamic analyses, we can generate a comprehensive system call whitelist for the initialization phase, termed the Initialize Seccomp File (I-SF). This I-SF encompasses the system call requirements of the program under various scenarios. Such a whitelist can more accurately reflect the program's runtime behavior, thereby playing a crucial role in subsequent

filter rule designs. The integration of these two phases provides a more comprehensive and accurate insight into system call requirements, offering robust support for subsequent filter rule designs.

C. Running Phase System Call Filter Set Generation Mechanism

In previous studies, it was observed that by monitoring the progression of container runtime, many applications within containers typically transition to their running phase after a certain duration. However, this approach might lead to misjudgments or may not be applicable to the majority of container images.

To accurately distinguish between the initialization and running phases of a containerized application (e.g., `nginx`), we adopt a hybrid approach that combines coarse-grained process behavior recognition with fine-grained system call analysis. Initially, we leverage internal service indicators such as the startup of the `docker-entrypoint.sh` process and its child services to approximate the service startup boundary. Specifically, when Docker configures permissions and initializes container metadata, and during the early stages of `nginx` service startup, we regard the container as being in the initialization phase. Once the `nginx` service is fully initialized and begins waiting for external connections, we consider it to have entered the running phase.

To refine this transition point, we collect system calls using `eBPF` and perform preprocessing by segmenting the call sequences into fixed-size time windows (e.g., 0.001 seconds). As shown in Fig. 2, for each time window, a list of system calls $\{x_{i-1}, x_{i-2}, \dots, x_{i-N}\}$ is extracted. These lists are then vectorized to generate feature vectors for each window, incorporating metrics such as syscall frequency and syscall types.

We then apply a similarity metric (e.g., cosine similarity) to compute the resemblance between consecutive time windows. Empirically, the feature distribution in the initialization phase differs significantly from that in the running phase. By setting a similarity threshold of 0.8, we scan through a sliding window (e.g., 5 seconds) to detect when similarity across windows stabilizes above the threshold. The final time window that meets this condition is designated as the segmentation point T , which marks the transition into the running phase.

With this segmentation point T , we employ `eBPF` to collect system calls executed by the container beyond this point. By collecting data over a sufficiently extended runtime, we obtain the Running Seccomp File (R-SF). This R-SF can then be utilized to reinforce restrictions, monitoring, and alerts for the container during its operational phase.

D. Phased Filtering and Monitoring Mechanism

In this section, as Fig. 3 shows, we will delve into the design of the phased filtering and monitoring mechanism. These rules, derived from both static and dynamic analyses, aim to generate refined system call filtering policies for containers. This refinement seeks to reduce the attack surface of containers and enhance the overall security of the system.

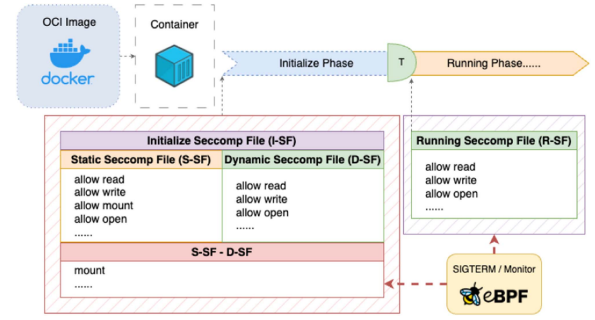


Fig. 3. Phased filtering and monitoring mechanism.

1) *Handling Forbidden System Calls:* By combining the system call filtering sets obtained from static analysis, S-SF, and dynamic analysis, D-SF, we can derive the initialization phase system call policy set, I-SF.

Terminating Processes with SIGTERM: If a process invokes a system call that is not present in the initialization phase system call policy set, S-SF, we can adopt a conservative approach by using the SIGTERM signal to terminate the running process and log an alert. This helps mitigate potential risks posed by unfamiliar system calls.

2) *System Calls to Handle Dynamic Analysis:* The results of dynamic analysis reflect the actual system call behavior during program execution.

- **Permitting Uncovered System Call Execution from Dynamic Analysis:** If a system call is only found in the static analysis system call policy set, S-SF, but not in the dynamic analysis system call policy set, D-SF, it might be considered an exceptional case, possibly because the dynamic analysis did not trigger that call. In such scenarios, the system call can be allowed to proceed, but with a deeper analysis using `eBPF` to understand its detailed parameters and behavior, followed by alerting.
- **Allowing System Calls Matched by the Dynamic Configuration File:** If a system call is matched within the dynamic analysis system call policy set, D-SF, irrespective of its presence in the static configuration file, it can be allowed to execute. Since the dynamic configuration file mirrors the program's actual runtime behavior, permitting these system calls can enhance system performance. In this case, detailed logging and alerting for these system calls can be reconsidered, opting only for event logging to reduce the performance overhead of security monitoring.

3) *Handling System Calls During the Runtime Phase:* The system call policy filter set for the runtime phase, R-SF, is relatively more restrictive compared to the initialization phase's system call policy filter set, Si. The difference set, I-SF - R-SF, represents the additional system calls used during the initialization phase. For such system calls, more detailed monitoring and alerting are essential. These system calls typically involve operations like `fork()` and `mount()` during initialization, which are usually not performed during the runtime phase. If they do occur at runtime, it could indicate malicious activities by attackers. However, outright prohibiting the system calls in the difference set, I-SF - R-SF, might lead to operational

TABLE III
IMAGE CATEGORY

Field	Image
Database	Redis, MySQL
Infrastructure	Nginx, Apache
Programming Language	Python, Java, Golang
Operating System	Ubuntu, CentOS, Fedora

issues if there are oversights in collecting the system call filter sets.

We have devised two strategies to handle the system calls from I-SF - R-SF: prohibition and alerting.

- **Prohibition of System Calls from I-SF - R-SF:** If the security operations team is sufficiently familiar with the application and is certain that these system calls will never be invoked during runtime, they can be deemed prohibited. Using eBPF to capture events of such system calls, a SIGTERM signal can be employed to terminate the running process and log the alert.
- **Alerting for System Calls from I-SF - R-SF:** If the security operations team is uncertain about whether a specific system call might be invoked with a very low probability during the runtime phase, a more granular monitoring approach is warranted. Using eBPF, events and associated parameters of such system calls can be captured. These event details can then be pushed to the monitoring platform for alerting. This will prompt the security operations team to manually assess whether the container has potential security vulnerabilities. With sufficient experience and insights gained from these alerts, a decision can later be made on whether to outright prohibit such system calls.

V. EXPERIMENT

A. Experimental Setup

To delve deeper into the performance and reliability of container technology across various domains, we conducted experiments on a computer equipped with an Intel i5-7300 processor. The computer ran Ubuntu 18.04 as its operating system, with a Linux kernel version of 4.10. Furthermore, for consistency and accuracy in our experiments, we opted for Docker version 20.10.24.

Before commencing the experiments, we conducted an in-depth analysis of resources available on DockerHub. From this analysis, we identified 10 popular official images with download counts exceeding 10 million. These images are widely recognized and utilized within the Docker community, highlighting their significance and value in practical application scenarios.

As Table III shows, these 10 popular images predominantly span four major functional domains:

- **Databases:** These images offer a variety of popular database management systems such as MySQL, PostgreSQL, and MongoDB, catering to data storage and management requirements.
- **Infrastructure:** services like Nginx, Redis, etc., which act as pillars supporting other applications or services.
- **Programming Languages:** These images encompass runtime environments and dependency libraries for various

TABLE IV
NUMBER OF SYSTEM CALLS IN EACH PHASE

Image	S-SF	D-SF	I-SF	R-SF
Redis:4.0.2	130	77	134	69
MySQL:8.3.0	184	118	186	111
Nginx:1.24.0	124	98	129	92
httpd:2.4.39	146	109	146	105
Python:3.9.18	134	102	134	101
Java:8-jdk-alpine	154	110	159	104
Golang:1.20.14	122	102	123	102
Ubuntu:9.18-22.10_beta	218	167	219	154
CentOS:centos7	210	154	211	153
Fedora:31	215	161	216	156

programming languages such as Python, Node.js, etc., facilitating developers in application development and deployment.

- **Operating Systems:** Images in this category provide lightweight operating system environments, like Alpine Linux, tailored for running specific applications or services.

By categorizing these images meticulously, our goal is to gain a comprehensive understanding of the performance and characteristics of different types of container images in specific application scenarios. This foundational knowledge will pave the way for subsequent experiments and research endeavors.

B. Comparison of System Call Shrinkage Effects

Through meticulous static analysis, we have successfully crafted a more precise and stringent system call filtering strategy. Compared to the initial extensive list of system calls, we have streamlined and refined it, making the strategy more accurate and efficient. As shown in Fig. 4 and Table IV, the average number of system calls always remains around 140 in the analyzed images. However, it is noteworthy that operating system images exhibit a more intricate and extensive reliance on system calls. This complexity is reflected in their range of system calls, which typically fluctuates between 210 and 218. In contrast, images from other categories fall between 82 and 184 system calls. This further underscores our efforts in deep analysis and optimization of system calls.

During dynamic analysis, we did identify some new system calls. While the count of these calls is relatively minimal, when combined with the static analysis-derived strategy set S-SF, we have successfully established a more comprehensive and holistic strategy set I-SF. Notably, during runtime, there's a general decline in the number of system calls in R-SF. This further validates the effectiveness and necessity of enhancing system call restrictions to bolster security.

To better illustrate the impact of syscall reduction, we analyzed the syscall sets between the initialization phase and runtime phase. During initialization, containers often perform various setup tasks, such as loading dependencies, writing configuration files, initializing sockets, and spawning subprocesses, all of which demand a broader range of system calls [43]. In contrast, the runtime phase exhibits more consistent and constrained behavior. As demonstrated in our evaluation (e.g., Redis's syscall set reduced from 134 to 69), this reduction

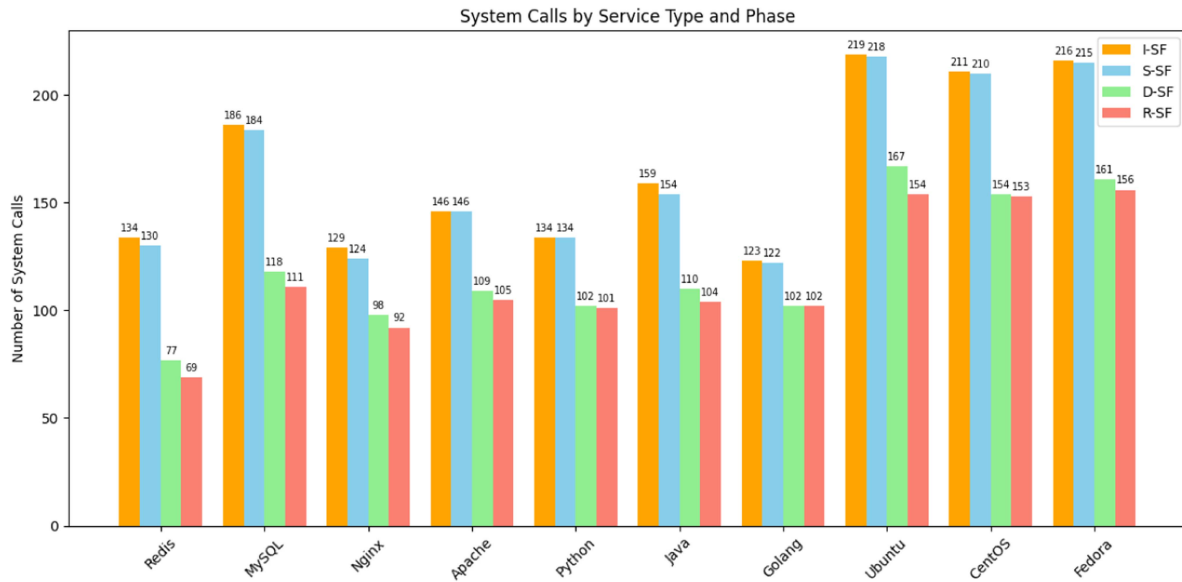


Fig. 4. System calls by service type and phase.

TABLE V
MISSING SYSTEM CALLS PER CONTAINER IMAGE

Container Image	Missing System Calls	Count
nginx	wait4, epoll_create1, forkf, statfs, pipe, io_setup, getrlimit	7
httpd	chmod, getrandom, rename, unlinkat, vfork	5
memcached	—	0
redis	epoll_create1, fork	2

significantly narrows the attack surface. By enforcing the refined R-SF whitelist during the runtime phase, the PSF method reduces the likelihood of exploitation by eliminating unnecessary system call exposure, thereby enhancing the security posture of containerized applications.

In summary, this in-depth and comprehensive analysis of system calls has not only provided us with profound insights into the initialization strategy set I-SF but also equipped us with greater flexibility and adaptability during actual runtime phases. The refinement and implementation of this strategy have significantly elevated the overall security and reliability of the system, offering users a safer and more efficient computing environment.

C. System Call Differences Analysis

We selected four representative container images (*nginx*, *httpd*, *memcached*, and *redis*) and used automated testing tools to conduct functional testing. During the process, we collected system calls during both the initialization and runtime phases, and monitored crash information to identify abnormal behaviors or program errors caused by missing system calls. The experimental results are shown in Table V.

By comparing the system calls captured in these two phases, we observed that some syscalls appeared in the initialization profile but were still triggered at runtime. Table V presents the system calls that were missing during runtime and their corresponding counts for each container.

As shown in the table, containers like *nginx* and *httpd* exhibited a relatively high number of missing system calls, with 7 and 5 respectively, while *redis* had 2 and *memcached* had none. Further analysis of crash logs indicated that missing critical syscalls such as *epoll_create1* and *fork* could lead to service exceptions or functional failures.

These observations demonstrate that there exists a clear behavioral discrepancy between the initialization and runtime phases. Relying solely on syscall profiles collected during initialization may lead to incomplete security policies that fail to accommodate actual runtime needs.

This validates the necessity and practical significance of our proposed **phase-aware system call extraction mechanism**, which takes into account the differences in syscall behavior across phases to generate more accurate and robust SECCOMP profiles for containerized environments.

D. Comparative Analysis of Container and Program Calls

To evaluate the effectiveness of the PSF system in static system call analysis and whitelist generation, we compared the system's performance in analyzing system calls executed within container environments. Additionally, to assess the accuracy of the method in analyzing the correctness of system call results, we referred to the test cases provided by ChestNut and employed its patching mechanism to constrain program system calls, thereby verifying the precision of our approach.

1) *Number of System Calls in the Whitelist*: Temporal Specialization(TS) [46] is an innovative methodology that dynamically constrains the availability of system calls to a process according to its distinct execution phases. As shown in the table VI, the number of system calls included in the whitelist generated through source code analysis is approximately 100, which is significantly fewer than the 357 system calls permitted by Docker's default Seccomp configuration. However, since this method only analyzes the source code of the core application, it fails to capture additional system calls

TABLE VI
COMPARISON OF SYSTEM CALL WHITELISTS

Image	Method	Whitelisted	Missing	Filtered	Total
nginx	TS	106	12	239	357
	PSF	155	—	202	357
httpd	TS	94	12	251	357
	PSF	140	—	217	357
memcached	TS	99	9	249	357
	PSF	128	—	229	357
redis	TS	90	13	254	357
	PSF	138	—	219	357

TABLE VII
SYSTEM CALLS OF CHESTNUT AND PSF

Method	Test	Simple	Test2_dynamic
ChestNut-Binalyzer	43	2	268
ChestNut-Dynalyzer	9	3	14
PFS-Dynalyzer	9	3	14

related to the container runtime environment. Manual analysis of the container's functionality after applying the whitelist via Seccomp revealed that at least 9 to 13 system calls were missing. Some of these calls are essential for initializing the container environment, while others are required by command-line tools. In contrast, the system call whitelist generated in this study is based on a more comprehensive analysis tailored to the container environment. To address the issue of missing system calls in static analysis, our approach supplements the statically derived whitelist with results from dynamic analysis.

It was observed that the increase in the number of system calls in the whitelist, after the integration of static and dynamic analysis, varied across different types of containers. For instance, network service containers such as nginx and httpd exhibited a significant increase in system calls supplemented through dynamic analysis. This is primarily due to their broader functional scope, reliance on dynamic libraries, and the presence of multiple command-line tool binaries. In contrast, data caching containers showed a relatively smaller increase, with memcached presenting the least number of additional system calls. A comprehensive analysis of its binary files and functional characteristics reveals that memcached includes fewer binaries and is primarily designed for in-memory caching. Unlike redis, a similar caching service that supports data persistence through snapshotting (RDB) and append-only file (AOF) mechanisms, memcached does not support persistent storage. Consequently, its use of system calls is more limited, resulting in fewer additions during dynamic analysis.

2) *Number of Calls in Program Analysis:* To further validate the applicability and accuracy of the PSF method in practical containerized environments, we selected its dynamic analysis module to evaluate its capability of capturing runtime system calls. In this experiment, we introduced the Chestnut [47], which is an automated sandboxing tool based on system call-level analysis and supports both static and dynamic analysis modes.

However, the PSF method does not involve the compilation or linking process of applications. Its static analysis primarily focuses on the behavioral characteristics of programs running within containers, which differs from the static analysis mechanism employed by Chestnut. Therefore, we emphasize the

comparison of the dynamic detection results between PSF and Chestnut to evaluate the accuracy and completeness of PSF in capturing essential system calls in realistic container environments. For this purpose, we selected three test programs: statically compiled **Test** and **Simple**, as well as dynamically compiled **Test2_dynamic**.

As shown in Table VII, the system calls obtained by PSF-Dynalyzer are consistent with those found by ChestNut-Dynalyzer. Furthermore, running the tested programs with only the system calls identified by PSF did not result in any errors, confirming that the captured call set is sufficient for correct execution. This supports the accuracy and practicality of our method. We also observed a noticeable difference between the system calls found by static and dynamic analyses. In statically compiled programs, library functions are often excluded, resulting in a much smaller number of detected calls. For example, in the **Simple** program, static analysis even missed necessary system calls, emphasizing the necessity of dynamic analysis. Additionally, in the dynamically compiled **Test2_dynamic** program, ChestNut-Binalyzer reported 268 system calls. This large number stems from including calls from libraries like `libc.so.6`, which are not all actually used during execution. In contrast, the dynamic methods—both ChestNut-Dynalyzer and PSF-Dynalyzer—identified only 14 calls, accurately reflecting the true behavior of the application.

VI. CONCLUSION

The Phased Syscall Filter (PSF) presented herein stands as a sophisticated system call filtration framework meticulously crafted to augment the constraints on system call utilization within containerized environments during their operational phases. Through a judicious amalgamation of both static and dynamic analytical methodologies, this innovative framework curates an exhaustive whitelist of permissible system calls, ensuring comprehensive coverage and minimalistic access permissions. The strategic integration and adept utilization of eBPF technology serve as a pivotal enabler, substantially curtailing the performance overheads emanating from incessant transitions between the distinct realms of user and kernel spaces. Furthermore, the deployment of an advanced container segmentation point identification algorithm bestows upon the framework the capability to discern and demarcate the intricate nuances between initialization and runtime phases. This granular differentiation propels the formulation and deployment of stratified system call filtration paradigms, wherein containers are subjected to nuanced, phase-specific system call restrictions, thereby optimizing operational integrity and security postures. In summation, PSF emerges as an avant-garde system call filtration solution, distinguished by its intricate granularity and pervasive monitoring mechanisms, fortifying the holistic security and surveillance frameworks enveloping containerized infrastructures.

PSF is designed with scalability and adaptability as core principles. Its system call filtering mechanism operates independently within each container instance, without relying on shared or global state, enabling it to scale seamlessly with the number of

containers. Upon container startup, PSF automatically generates a system call whitelist based on observed initialization behavior and enforces corresponding runtime restrictions. As it requires neither persistent monitoring nor manual per-container configuration, PSF remains lightweight and well-suited for high-churn environments.

REFERENCES

- [1] X. Lin, L. Lei, Y. Wang, J. Jing, K. Sun, and Q. Zhou, "A measurement study on linux container security: Attacks and countermeasures," in *Proc. 34th Annu. Comput. Secur. Appl. Conf.*, 2018, pp. 418–429.
- [2] M. Kerrisk, "Namespaces in operation, part 1: Namespaces overview," Website, 2013. [Online]. Available: <https://lwn.net/Articles/531114>
- [3] N. Brown, "Control groups, part 1: On the history of process grouping," Website, 2014. [Online]. Available: <https://lwn.net/Articles/603762>
- [4] C. V. Stephen Smalley and W. Salamon, "Implementing selinux as a linux security module," Website, 2001. [Online]. Available: <https://www.cs.unibo.it/sacerdot/doc/so/slm/selinux-module.pdf>
- [5] C. Cowan, S. Beattie, G. Kroah-Hartman, C. Pu, P. Wagle, and V. Gligor, "{SubDomain}: Parsimonious server security," in *Proc. 14th Syst. Admin. Conf.*, 2000, pp. 355–368.
- [6] Linux, "Seccomp BPF (secure computing with filters), linux kernel documentation," Website, 2024. [Online]. Available: https://static.lwn.net/kernel/doc/userspace-api/seccomp_filter.html
- [7] Moby Organization, "moby/default.json at master of moby/moby," Website, 2024. [Online]. Available: <https://github.com/moby/moby/blob/master/profiles/seccomp/default.json>
- [8] D. Merkel, "Docker: Lightweight linux containers for consistent development and deployment," *Linux J.*, vol. 2014, Mar. 2014, Art. no. 239.
- [9] D. Bernstein, "Containers and cloud: From LXC to docker to kubernetes," *IEEE Cloud Comput.*, vol. 1, no. 3, pp. 81–84, Sep. 2014.
- [10] H. Lu et al., "Research on intelligent detection of command level stack pollution for binary program analysis," *Mobile Netw. Appl.*, vol. 26, pp. 1723–1732, 2021.
- [11] Z. Tian, M. Li, M. Qiu, Y. Sun, and S. Su, "Block-DEF: A secure digital evidence framework using blockchain," *Inf. Sci.*, vol. 491, pp. 151–165, 2019.
- [12] Z. Gu, W. Hu, C. Zhang, H. Lu, L. Yin, and L. Wang, "Gradient shielding: Towards understanding vulnerability of deep neural networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 921–932, Apr.–Jun., 2021.
- [13] I. Agadakis, D. Jin, D. Williams-King, V. P. Kemerlis, and G. Portokalidis, "Nibbler: Debloating binary shared libraries," in *Proc. 35th Annu. Comput. Secur. Appl. Conf.*, 2019, pp. 70–83.
- [14] X. Pang, Z. Wang, D. Liu, J. C. Lui, Q. Wang, and J. Ren, "Towards personalized privacy-preserving truth discovery over crowdsourced data streams," *IEEE/ACM Trans. Netw.*, vol. 30, no. 1, pp. 327–340, Feb. 2022.
- [15] E. G. Young et al., "The true cost of containing: A gVisor case study," in *Proc. 11th USENIX Workshop Hot Topics Cloud Comput.*, 2019.
- [16] A. Randazzo and I. Tinnirello, "Kata containers: An emerging architecture for enabling mec services in fast and secure way," in *Proc. 6th Int. Conf. Internet Things: Syst., Manage. Secur.*, 2019, pp. 209–214.
- [17] Z. Li et al., "{RunD}: A lightweight secure container runtime for high-density deployment and high-concurrency startup in serverless computing," in *Proc. USENIX Annu. Tech. Conf.*, 2022, pp. 53–68.
- [18] A. Kurmus, A. Sornioti, and R. Kapitza, "Attack surface reduction for commodity os kernels: Trimmed garden plants may attract less bugs," in *Proc. 4th Eur. Workshop Syst. Secur.*, 2011, pp. 1–6.
- [19] Linux Kernel, "Kernel probes (kprobes)," Website, 2024. [Online]. Available: <https://docs.kernel.org/trace/kprobes.html>
- [20] S. Jang et al., "SecQuant: Quantifying container system call exposure," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2022, pp. 145–166.
- [21] OpenContainers, "runc," Website, 2024. [Online]. Available: <https://github.com/opencontainers/runc>
- [22] H. Lu, C. Jin, X. Helu, C. Zhu, N. Guizani, and Z. Tian, "AutoD: Intelligent blockchain application unpacking based on JNI layer deception call," *IEEE Netw.*, vol. 35, no. 2, pp. 215–221, Mar./Apr. 2020.
- [23] J. Hou, F. Liu, H. Lu, Z. Tan, X. Zhuang, and Z. Tian, "A novel flow-vector generation approach for malicious traffic detection," *J. Parallel Distrib. Comput.*, vol. 169, pp. 72–86, 2022.
- [24] M. Pan et al., "DHPA: Dynamic human preference analytics framework: A case study on taxi drivers' learning curve analysis," *ACM Trans. Intell. Syst. Technol.*, vol. 11, no. 1, pp. 1–19, 2020.
- [25] N. Hu, Z. Tian, H. Lu, X. Du, and M. Guizani, "A multiple-kernel clustering based intrusion detection scheme for 5G and IoT networks," *Int. J. Mach. Learn. Cybern.*, vol. 12, pp. 1–16, 2021.
- [26] Z. Wang, K. Liu, J. Hu, J. Ren, H. Guo, and W. Yuan, "Attrleaks on the edge: Exploiting information leakage from privacy-preserving co-inference," *Chin. J. Electron.*, vol. 32, no. 1, pp. 1–12, 2023.
- [27] S. Ghavamnia, T. Palit, A. Benameur, and M. Polychronakis, "Confine: Automated system call policy generation for container attack surface reduction," in *Proc. 23rd Int. Symp. Res. Attacks Intrusions Defenses*, 2020, pp. 443–458.
- [28] N. DeMarinis, K. Williams-King, D. Jin, R. Fonseca, and V. P. Kemerlis, "Sysfilter: Automated system call filtering for commodity software," in *Proc. 23rd Int. Symp. Res. Attacks, Intrusions Defenses*, 2020, pp. 459–474.
- [29] H. Lu, C. Jin, X. Helu, X. Du, M. Guizani, and Z. Tian, "DeepAutoD: Research on distributed machine learning oriented scalable mobile communication security unpacking system," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 2052–2065, Jul./Aug. 2022.
- [30] S. Pailoor, X. Wang, H. Shacham, and I. Dillig, "Automated policy synthesis for system call sandboxing," in *Proc. ACM on Program. Lang.*, vol. 4, no. OOPSLA, pp. 1–26, 2020.
- [31] Linux Manual Page, "Capabilities," Website, 2024. [Online]. Available: <https://man7.org/linux/man-pages/man7/capabilities.7.html>
- [32] S. Ghavamnia, T. Palit, S. Mishra, and M. Polychronakis, "Temporal system call specialization for attack surface reduction," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 1749–1766.
- [33] strace, "strace," Website, 2024. [Online]. Available: <https://github.com/strace/strace>
- [34] Draios, "Sysdig," Website, 2024. [Online]. Available: <https://github.com/draios/sysdig>
- [35] eBPF, "ebpf," Website, 2024. [Online]. Available: <https://ebpf.io>
- [36] Slimtoolkit, "Slim," Website, 2024. [Online]. Available: <https://github.com/slimtoolkit/slim>
- [37] L. Lei et al., "Speaker: Split-phase execution of application containers," in *Proc. Detection Intrusions Malware, Vulnerability Assessment: 14th Int. Conf.*, Bonn, Germany, 2017, pp. 230–251.
- [38] M. Pancholi, A. D. Kellas, V. P. Kemerlis, and S. Sethumadhavan, "Timeloops: System call policy learning for containerized microservices," 2022, *arXiv:2204.06131*.
- [39] Google, "Addresssanitizer," Website, 2024. [Online]. Available: <https://github.com/google/sanitizers>
- [40] T. Iiguni, H. Kamei, and K. Saisho, "Sprofiler: Automatic generating system of container-native system call filtering rules for attack surface reduction," in *Proc. IEEE Int. Conf. Comput. Sci. Comput. Intell.*, 2021, pp. 704–709.
- [41] S. Kim, B. J. Kim, and D. H. Lee, "Prof-gen: Practical study on system call whitelist generation for container attack surface reduction," in *Proc. IEEE 14th Int. Conf. Cloud Comput.*, 2021, pp. 278–287.
- [42] X. Wang, Q. Shen, W. Luo, and P. Wu, "RSDS: Getting system call whitelist for container through dynamic and static analysis," in *Proc. IEEE 13th Int. Conf. Cloud Comput.*, 2020, pp. 600–608.
- [43] J. Jiang et al., "A dynamic ensemble algorithm for anomaly detection in IoT imbalanced data streams," *Comput. Commun.*, vol. 194, pp. 250–257, 2022.
- [44] Z. Zhong, Z. Yu, Y. Yang, W. Wang, and K. Yang, "Patchad: A lightweight patch-based MLP-mixer for time series anomaly detection," 2024. [Online]. Available: <https://arxiv.org/abs/2401.09793>
- [45] V. L. Rajagopalan, K. Kleftogiorgos, E. Göktas, J. Xu, and G. Portokalidis, "Syspart: Automated temporal system call filtering for binaries," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2023, pp. 1979–1993, doi: [10.1145/3576915.3623207](https://doi.org/10.1145/3576915.3623207).
- [46] S. Ghavamnia, T. Palit, S. Mishra, and M. Polychronakis, "Temporal system call specialization for attack surface reduction," in *Proc. 29th USENIX Secur. Symp. (USENIX Secur. 20)*, USENIX Assoc., 2020, pp. 1749–1766. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity20/presentation/ghavamnia>
- [47] C. Canella, M. Werner, D. Gruss, and M. Schwarz, "Automating seccomp filter generation for linux applications," in *Proc. Cloud Comput. Secur. Workshop*, New York, NY, USA, 2021, pp. 139–151, doi: [10.1145/3474123.3486762](https://doi.org/10.1145/3474123.3486762).



Ke Chen is currently working toward the PhD degree with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing, China. His research focuses on network offense and defense and the application of AI in network attack and defense.



Binxing Fang is currently a professor and PhD supervisor with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangdong Province, China. His research interests include computer networks and cyberspace security.



Hui Lu received the BE degree in electronic science and technology and the MS degree in optical engineering from Southwest Jiaotong University, Chengdu, China, in 2003 and 2006, respectively, and the PhD degree in electromagnetic field and microwave technology from the Beijing University of Posts and Telecommunications, Beijing, China, in 2010. He is currently a professor with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China. From 2010 to 2017, he was a research associate with the Institute of Microelectronics, Chinese Academy of Sciences, Beijing, China. In 2017, he joined Guangzhou University, Guangzhou, China. His research interests include cyberspace security and intelligent attack-defence.

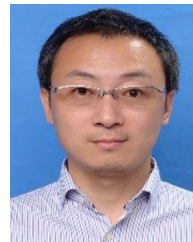
croelectronics, Chinese Academy of Sciences, Beijing, China. In 2017, he joined Guangzhou University, Guangzhou, China. His research interests include cyberspace security and intelligent attack-defence.



Yuan Liu (Senior Member, IEEE) received the BSc degree in the Honor School, Harbin Institute of Technology, China, in 2010, and the PhD degree in the School of Computer Engineering from Nanyang Technological University (NTU), Singapore, in 2014. She is a professor with the Cyberspace Institute of Advanced Technology of Guangzhou University in Guangdong, China. She was an associate professor with northeastern university, China from 2015 to 2022. From 2014 to 2015, and she ever worked as research fellow with Joint NTU-UBC Research Center of Excellence in Active Living for the Elderly (LILY), NTU, Singapore. Her research interests include incentive mechanism design, federated learning, trust management, blockchain consensus protocols, blockchain powered artificial intelligence, and threat intelligence system.



Yinnan Yao graduated from the School of Cyberspace Security, Guangzhou University, Guangzhou, China, with a master's degree in computer technology. His research focuses on Linux container security.



Zhihong Tian received the BS, MS, and PhD degrees in computer science and technology from the Harbin Institute of Technology, Harbin, China, in 2001, 2003, and 2006 respectively. He is currently a professor, and vice president with Guangzhou University, Guangdong Province, China. He is honored as Pearl River Scholar in Guangdong Province. He is also a part-time professor with Carlton University, Ottawa, Canada. previously, he served in different academic and administrative positions with the Harbin Institute of Technology. He has authored more than 200 journal and conference papers. His research interests include computer networks and cyberspace security. His research has been supported in part by the National Natural Science Foundation of China, National Key research and Development Plan of China, National High-tech R&D Program of China (863 Program). He also served as a member, chair, and general chair of a number of international conferences. He is a distinguished member of the China Computer Federation.